

Healthcare Triage Assistant

How This Project Was Built — Hackathon Explainer Notes

An AI-primary, human-in-the-loop, multimodal triage-support tool for government hospitals, PHCs, public health camps, company clinics, industrial-estate health units, and campus health centers — built to run fully offline on CPU-only hardware.

■ Non-diagnostic, educational hackathon project

This system never gives a final diagnosis or prescribes treatment. It organizes information and highlights urgency signals for a qualified human reviewer to confirm. Only synthetic / demo data is used with this build — never real patient data.

Prepared as reference notes for explaining the project's design, architecture, and engineering decisions to hackathon judges.

168 automated tests • zero-crash pipeline audit • offline CPU-only inference

Contents

1. Problem Statement
2. Solution Overview — The One-Sentence Pitch
3. System Architecture
4. Technology Stack
5. Key Design Decisions (and why they matter)
6. How Each Major Feature Works
7. The Pre-Hackathon Pipeline Audit — an Engineering Story
8. Testing & Verification
9. Known Limitations (said out loud, on purpose)
10. Hackathon Talking Points & Anticipated Judge Q&A;
11. 2-Minute Live Demo Script

1. Problem Statement

Frontline health workers across India — at PHCs, government hospitals, public health camps, company clinics, industrial-estate health units, and campus health centers — often need to triage patients quickly with limited staff, limited connectivity, and patients who may describe symptoms in Hindi, Odia, or English, sometimes by voice rather than text. A useful triage-support tool needs to:

- Take in patient-reported symptoms in multiple formats (typed text, voice, photos of visible conditions, photos or scans of lab reports/prescriptions).
- Extract structured, usable information from unstructured, sometimes messy free text.
- Flag urgency accurately — the single highest-stakes requirement, because under-triaging a genuine emergency is dangerous and over-triaging everything makes the tool useless.
- Work fully offline on modest, CPU-only hardware, since many of these facilities cannot guarantee reliable internet access.
- Support India's linguistic diversity, not just English.
- Keep a human reviewer firmly in control — never replace clinical judgement.
- Protect patient privacy given how sensitive health data is.

That is the brief this project answers.

2. Solution Overview — The One-Sentence Pitch

The pitch

A Flask web app where a health worker records a patient's symptoms (text, voice, or a photo/scan), a local offline AI model reads and structures that information, a deterministic rules engine independently checks for red-flag symptoms and vitals, the two are combined so the AI can only ever raise urgency and never lower it, and a human reviewer sees the resulting risk tier, the AI's reasoning, and a rules-only cross-check — before signing off, adding notes, and exporting a referral PDF if the patient needs to be sent to a higher facility.

The core insight the whole design hangs on:

AI is genuinely useful for reading messy human language, but should never be the thing that decides how urgent a patient is. This project treats those as two separate jobs done by two separate components — language understanding (the AI model) and safety-critical decision-making (a deterministic, auditable rules engine) — and architecturally guarantees the AI can only add caution, never remove it.

3. System Architecture

The diagram below is the single most important thing to be able to draw on a whiteboard for judges — it is the whole safety argument in one picture.

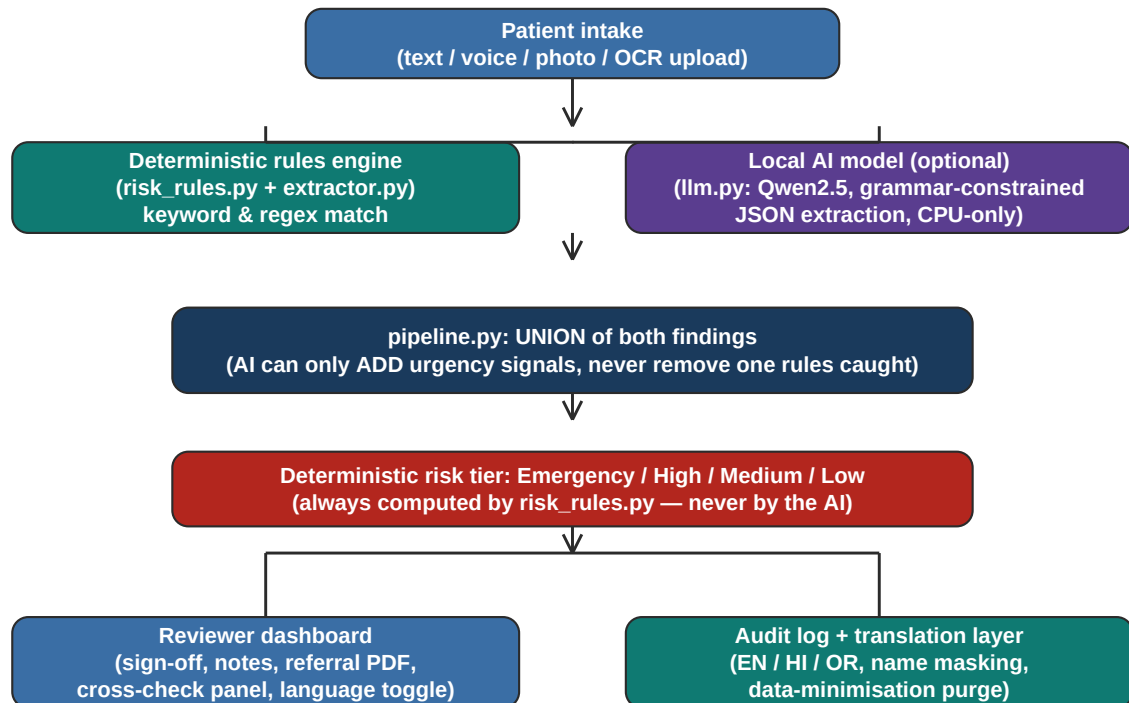


Figure 1. Request flow from patient intake to reviewer sign-off. Note that risk tier (the red box) is always computed by the deterministic engine — the AI box feeds into it, never replaces it.

3.1 Module map

Module	Responsibility
app.py	Flask routes/controllers, security wiring (CSRF, sessions, rate limiting), error handling.
config.py	Environment-driven configuration for dev / prod / testing profiles.
wsgi.py, gunicorn.conf.py	Production WSGI entrypoint and server tuning.
triage_engine/risk_rules.py	Symptom ontology, negation handling, vitals plausibility bounds, deterministic risk scoring — the safety net.
triage_engine/extractor.py	Free text → structured note (the deterministic fallback path, used when AI is unavailable).
triage_engine/pipeline.py	Orchestrator: runs AI extraction when available, unions its findings with the rules engine's, and always computes the final tier via the deterministic engine.
triage_engine/llm.py	Local, offline, CPU-only text model (Qwen2.5-1.5B via llama-cpp-python): grammar-constrained JSON extraction, narrative summary, follow-ups, translation, output-side safety filtering.
triage_engine/vision.py	Local vision models (moondream2 default; MedGemma-4B opt-in) for photo description / hedged possible-findings.
triage_engine/ocr.py	Image/PDF → text (pytesseract + pdf2image) with a usable-text guard.
triage_engine/translator.py	EN/HI/OR phrase-bank translation for UI strings, symptom names, categories, follow-up questions — deterministic and instant.
triage_engine/indic_translate.py	Dedicated English→Hindi/Odia MT model (IndicTrans2), preferred over the generalist chat model for translating free-form AI text.
triage_engine/referral.py	Structured note → referral PDF (reportlab).
triage_engine/audit.py	Audit logging and patient-name masking helpers.
triage_engine/database.py	SQLite schema, WAL mode, context-manager transactions, lightweight auto-migration.
templates/, static/	Server-rendered UI — Jinja2 + vanilla JS + hand-rolled CSS, no build step, no CDN dependency.
tests/	168-test automated suite covering safety-critical logic end to end.

No network calls happen at runtime. OCR, risk scoring, extraction, AI extraction/summarization, translation, and vision description all run locally on CPU. The only network use anywhere in the project is the one-time model download.

4. Technology Stack

Layer	Choice
Web framework	Flask (server-rendered, Jinja2 templates)
Database	SQLite (WAL mode, auto-migrating schema)
Deterministic extraction	Python regex / keyword ontology (triage_engine/risk_rules.py)
Local text AI model	Qwen2.5-1.5B-Instruct, quantized GGUF (Q4_K_M, ~1GB), served via llama-cpp-python, grammar-constrained (GBNF) JSON output
Local vision AI models	moondream2 (default, fast, descriptive captions) and MedGemma-4B-IT (opt-in, medical-domain hedged findings), both via llama-cpp-python
Machine translation	IndicTrans2 (AI4Bharat, via transformers/torch) for English→Hindi/Odia, preferred over the generalist chat model; a hand-built phrase bank for fixed UI strings
OCR	pytesseract + pdf2image (poppler) for lab reports/prescriptions, images and PDFs
Voice input	Browser-native Web Speech API (client-side, no server dependency), following the same language selector as the rest of the UI
PDF generation	reportlab (referral export)
Security	werkzeug password hashing, hand-rolled CSRF middleware, custom sliding-window login rate limiter, security headers, hardened session cookies
Deployment	gunicorn + wsgi.py, Dockerfile + docker-compose.yml, environment-driven config
Testing	Python unittest / pytest — 168 tests, ~10 seconds, zero external services required

Why these choices? Every model is small enough to run on CPU-only hardware with no GPU and no internet after the one-time download — a hard requirement for the target facilities. SQLite and server-rendered Flask avoid any infrastructure a low-resource deployment can't support. reportlab and pytesseract are dependency-light, offline-friendly libraries with no external API calls.

5. Key Design Decisions (and why they matter)

5.1 — The AI can escalate risk, but never downgrade it

This is the single most important design decision in the whole project, and the one to lead with when explaining the safety architecture. **pipeline.py** runs the deterministic rules engine and the AI extraction independently, then takes the **union** of the red-flag symptoms each one found. The final risk tier is *always* computed by the deterministic engine from that union — the AI's output can add a red flag the keyword engine missed, but it can never make one disappear. This guarantee has its own dedicated automated regression test (`test_pipeline.py::test_ai_cannot_downgrade_a_rules_detected_red_flag`), and the reviewer can see both numbers side by side via an expandable cross-check panel on every AI-powered note.

5.2 — Deterministic risk-tiering is a safety decision, not a shortcut

It would be technically simpler to let the AI model output a risk tier directly. This project deliberately does not do that: a small local language model is excellent at reading and organizing free text, but is not something you want deciding whether a patient needs emergency care, because its failure modes are opaque and hard to audit. Rules engines are boring, auditable, and testable — exactly the properties you want for a safety-critical decision.

5.3 — Everything runs offline, on CPU only

Target facilities cannot guarantee internet access. Every model (text extraction, vision, translation) is downloaded once and then runs entirely locally — no API keys, no outbound calls at request time. The app also works completely without any model downloaded at all, falling back transparently to the deterministic rules-only pipeline with a clear on-screen note, so a demo never “breaks” for lack of a GPU or connectivity.

5.4 — The AI never names a diagnosis or prescribes anything

Every AI output — summary, extraction, translation — passes through an output-side safety filter before it is ever shown to a reviewer, which rejects and falls back to the deterministic path if the model's raw output contains prescription/dosage phrasing or a direct diagnosis statement (e.g. “The diagnosis is pneumonia”, “Take 500 mg paracetamol every 6 hours”). The one narrow, deliberate exception is the opt-in MedGemma photo-findings feature, which is scoped tightly (hedged language only, explicitly “possible considerations for the reviewer to confirm”, never a final answer) precisely because it is the one place a hedged differential is genuinely useful.

5.5 — Privacy and data minimisation by design

- A mandatory consent screen before every intake.
- Patient name is optional; every patient is referenced by a generated Triage ID, not by name, in the reviewer workflow.
- The reviewer view masks patient names.
- Every view, edit, and export is written to an audit log.
- An auditor-only control purges closed records (data minimisation).
- Passwords are hashed, sessions are hardened, forms are CSRF-protected, and security headers are set on every response.

5.6 — Real multilingual support, not just a translated UI shell

Odia in particular has weak coverage in general-purpose chat models. Rather than relying on the same small model for both extraction and translation, this project uses a dedicated English→Indic machine-translation

model (IndicTrans2) for free-form AI-generated text, with script-mismatch and repetition-loop detection to catch degenerate output, falling back to the generalist model and then to English if needed — and a hand-built phrase bank for the fixed UI strings, symptom names, and categories that never needs a model call at all.

6. How Each Major Feature Works

6.1 Multilingual intake: text, voice, and photo

The intake form accepts typed text or voice input via the browser's built-in Web Speech API (no server-side speech model needed), and the recognition language automatically follows whichever language the worker selected for the note — English, Hindi, or Odia — so a patient can describe symptoms in their own language rather than being forced into English. A single upload field also accepts a photo or scan (lab report, prescription, or a photo of a visible symptom like a wound or rash).

6.2 Structured extraction: rules engine + AI, in parallel

The deterministic path (`extractor.py` + `risk_rules.py`) scans the combined text (typed symptoms + any OCR'd report text) against a symptom ontology with aliases, a negation detector (so “no chest pain” is correctly excluded, including compound cases like “denies fever or chest pain”), and regex-based vitals extraction with plausibility bounds (so a pain score like “12/10” is never misread as a blood pressure reading). If a local AI model is installed, it runs in parallel: its JSON output is grammar-constrained at the token level (a compiled GBNF grammar from a JSON Schema) so it is structurally guaranteed to parse, with one automatic retry at a wider token budget before falling back. Both results are then unioned as described in §5.1.

6.3 OCR for lab reports and prescriptions

Any uploaded image or PDF is run through Tesseract OCR. Extracted text is scanned for both vitals (blood pressure, SpO2, temperature, etc. — which can independently escalate a case, e.g. a low SpO2 reading on a slip) and symptom keywords, so a scanned referral note is treated exactly like typed text. A guard recognizes OCR's own “no usable text found” placeholder message and excludes it from ever being fed to extraction as if it were real content.

6.4 Vision models for photos of visible symptoms

For an uploaded photo (not a PDF), a local vision model additionally generates a plain-language, non-diagnostic description of what is visible (moondream2 by default). An opt-in medical-domain model (MedGemma-4B) can instead produce a hedged, clinician-facing list of possible visible considerations — always phrased as something for the reviewer to confirm, never a final answer, and this is the one deliberately narrow exception to §5.4's no-diagnosis rule.

6.5 Reviewer dashboard and escalation workflow

The dashboard queues notes by priority tier first, then age, with filters by status, facility, and tier, plus a dedicated panel for upcoming and overdue maternal-health/chronic-disease follow-ups. Every note shows whether it was AI-extracted or rules-based, the AI's narrative summary (if available), and an expandable rules-only cross-check so the reviewer can see the deterministic engine's independent judgement alongside the AI's. Sign-off requires an explicit reviewer action — nothing is auto-closed.

6.6 Referral PDF export and the audit trail

A one-click referral PDF (built with reportlab) captures the chief complaint, structured symptoms, vitals, AI summary, and reviewer notes for handoff to a higher facility. Every view, edit, and export action across every role is written to an audit log, and an auditor-only control can purge closed records to minimise how long data is retained.

7. The Pre-Hackathon Pipeline Audit — an Engineering Story

This section is worth telling judges directly: it shows the difference between a demo that happens to work on the happy path and a system that was actually stress-tested. Before final delivery, every module in `triage_engine/` and every route in `app.py` was read end-to-end looking specifically for anything that could crash, silently misclassify a case, or leak data across roles. Ten real, confirmed bugs were found and fixed — each with a new regression test so it cannot silently come back.

Correctness / safety bugs (affected what risk tier a case got)

Bug	Impact & fix
OCR text never scanned for symptoms	A scanned note saying “unconscious, severe bleeding” produced no red flag if the typed box was empty. Fixed: both text sources are now scanned together.
Over-eager substring symptom matching	“weakness” falsely matched the <code>stroke_signs</code> alias, “bleeding” falsely matched <code>severe_bleeding</code> — forcing false emergencies for routine complaints. Fixed: matching is now one-directional.
Blood pressure regex too permissive	A pain score (“12/10”) or a date (“15/11”) could be misread as a BP reading and flagged abnormal. Fixed: bounded to physiologically plausible values.
Age-unit handling incomplete	“18 months” was misread as under 1 year; “75 days” fell through to 75 years and was labeled elderly. Fixed: ages are normalized to fractional years.
Compound negation only excluded one item	“denies fever or chest pain” only excluded fever, leaving chest pain wrongly reported as present. Fixed: extended to “or / and / nor” chains without crossing a comma, “but”, or “however”.
Negated symptoms lost their translations	Both extraction paths broke the link between a negated symptom and its Hindi/Odia label. Fixed: negated symptoms now stay canonical until display time, like present symptoms.

Crash / reliability and security bugs

Bug	Impact & fix
Referral PDF crashed on ordinary text	An unescaped ‘<’ in a reviewer note (e.g. “improving<discharge planned”) raised an uncaught exception. Fixed: all free text is now XML-escaped before use.
Missing 413 (file too large) error page	Added a proper styled error page matching every other error code.
Open redirect on <code>/login's next=</code> parameter	A crafted link could redirect a user to an attacker-controlled site after login. Fixed: restricted to same-site relative paths only.
Intake workers could view any patient's detail page	Editing the note ID in the URL bypassed access control. Fixed: a new <code>created_by_user_id</code> column now scopes intake-worker access to their own submissions.

Two usability bugs were also fixed (the intake form losing all field values on a validation error, and a stale translation cache surviving an AI-summary regeneration). Full detail on all ten is in `README.md §11.12`. This work added 34 new regression tests, taking the suite from 134 to 168 tests, and was followed by a full manual smoke test of the entire user journey with zero failures.

8. Testing & Verification

Test file	Tests	What it locks in
test_risk_rules.py	25	Symptom ontology, negation (including compound cases), vitals plausibility, age-unit sensitivity
test_llm_safety.py	25	Grammar-constrained extraction, retry logic, output-side diagnosis/prescription safety filter
test_pipeline.py	17	AI + rules union safety guarantee (AI cannot downgrade risk), ontology matching precision, OCR-placeholder filtering
test_security.py	12	CSRF protection, password hashing & migration, rate limiting
test_app_integration.py	35	Full Flask request/response behavior with an isolated test database — auth, access control, form handling, redirects
test_ocr.py	6	OCR text extraction and usable-text detection
test_vision_diagnosis.py	25	Vision model gating, MedGemma hedged-findings safety scoping
test_indic_translate.py	10	IndicTrans2 script/repetition-quality checks and fallback chain
test_translator.py	7	Phrase-bank translation completeness across all symptoms/categories
test_referral.py	6	Referral PDF generation does not crash on unescaped free text

168 tests total, runtime ~10 seconds, zero external services required (python -m unittest discover -s tests or pytest). The most important single test in the suite is `test_pipeline.py::test_ai_cannot_downgrade_a_rules_detected_red_flag` — it is the automated proof of the core safety claim in §5.1.

9. Known Limitations (said out loud, on purpose)

A hackathon project that claims to have no limitations is less credible than one that names them precisely. Being able to state these clearly is itself a strong signal to judges.

- 1 **Demo-account role model, not a real identity system:** no self-service registration, MFA, or password reset flow — a full identity/security stack is out of scope for this stage.
- 2 **Neither vision model is trained on radiology** — both are deliberately scoped to external/visible symptoms and will not attempt a radiological read on an X-ray.
- 3 **The referral PDF and intake confirmation page are not translated** into Hindi/Odia, unlike the triage detail page.
- 4 **Concurrent requests translating the same note** could each overwrite the other's cache write — no data corruption, just a recomputed translation next time.
- 5 **Comma is deliberately not treated as a negation-list connector** — it is genuinely ambiguous between “another denied item” and “a new clause”, and the safer failure mode (occasionally under-widening a negation) was chosen over the more dangerous one (wrongly negating a real symptom).
- 6 **The experimental moondream2 vision path** could not be exercised end-to-end in the original build environment (no internet access there) and should be sanity-checked on a machine with internet before a live demo.

10. Hackathon Talking Points & Anticipated Judge Q&A

10.1 Lead with these three points

- The AI reads and organizes messy human language; a separate, deterministic, auditable rules engine decides urgency — and the AI can only ever add caution, never remove it (\$5.1).
- Everything runs fully offline on CPU-only hardware after a one-time model download — no API keys, no internet dependency at request time.
- The project went through a dedicated, documented pipeline audit that found and fixed ten real bugs with new regression tests for each — 168 tests total, ~10 seconds to run.

10.2 Anticipated questions

“What happens if the AI model gets it wrong?”

It can't make the case less urgent than the rules engine found it — the final tier is always the deterministic engine's decision on the union of both findings. A wrong AI extraction can, at worst, add an unnecessary flag for the reviewer to dismiss, never hide a real one.

“Why not just use ChatGPT / a cloud API?”

Target facilities can't guarantee internet access, and sending real patient data to a third-party API raises privacy concerns this project is specifically designed to avoid. Everything here runs locally after a one-time download.

“Isn't a 1.5B model too small to be reliable?”

That's exactly why extraction output is grammar-constrained at the token level (not just asked nicely in a prompt) so it structurally cannot produce invalid JSON, has an automatic retry with a wider token budget, and falls back transparently and visibly to the deterministic engine if it still fails — the app never breaks, and every fallback is logged and labeled on-screen.

“How do you know the safety guarantee actually holds?”

It's not just a design claim — there's a dedicated automated test for it (`test_ai_cannot_downgrade_a_rules_detected_red_flag`) that runs on every change.

“What about patient privacy?”

Consent is mandatory before intake, names are optional and masked in the reviewer view, patients are tracked by a generated Triage ID, every action is audit-logged, and there's a data-minimisation purge control for closed records.

“Why Odia specifically, not just Hindi and English?”

Odia is under-represented in most general-purpose language models. Rather than accept poor quality, the project integrates a dedicated English→Indic MT model (IndicTrans2) with output-quality checks (script mismatch, repetition-loop detection) specifically because the default chat model's Odia output was unreliable.

“What would you add with more time?”

A real identity/security stack (MFA, self-service accounts, per-facility isolation), translating the referral PDF and confirmation page, a shared cache store for multi-instance deployment, and validating the experimental vision path on internet-connected hardware.

11. 2-Minute Live Demo Script

1. **Sign in as worker1** → consent screen → intake form. Type or use the microphone button to describe symptoms (try switching the language selector to Hindi or Odia first to show voice input following the selected language). Submit and point out the instant Triage ID and priority tier.
2. **Sign in as reviewer1** → Dashboard. Point out the priority stat tiles and the queue sorted emergency-first regardless of arrival order.
3. Open the emergency demo case. Point out the AI-Extracted vs Rules-Based badge, the AI narrative summary, and the expandable rules-based cross-check panel — this is the moment to explain the union-safety-net design from §5.1. Switch the language toggle through English → Hindi → Odia.
4. Open the case with an attached lab slip and show the OCR-extracted vitals feeding directly into the risk score.
5. Change a note's status, add a reviewer note, and click Download Referral PDF.
6. **Sign in as auditor1** → Audit Log to show every action was recorded, then run Purge Closed Records to demonstrate data minimisation.
7. **If a judge asks about production-readiness:** open GET /health to show live AI-model performance stats, and mention hashed passwords, CSRF protection, and the 168-test suite.

For the full installation and setup guide, see `INSTALL.md` in the project package (includes OS-specific dependency installation, model download options, and a troubleshooting section).